DSC 550 - Data Mining

Manoj K Kola

Week 6 - Term Project Milestone 1: Data Selection and EDA

1. Import Required Libraries

This code sets up the environment for data analysis and visualization using Python. First, we import **pandas**, a powerful library for handling and manipulating structured data. Pandas provides efficient data structures such as DataFrames and Series, which make it easier to load, clean, and analyze datasets.

Next, we import **numpy**, a widely used library for numerical computation in Python. It allows us to perform array operations, execute mathematical calculations, and support a range of statistical functions.

For data visualization, we import **matplotlib.pyplot**, the foundational Python library for creating plots, charts, and other graphical representations of data. It provides flexibility for building both simple and complex visualizations. To complement this, we also import **seaborn**, a library built on top of matplotlib that enables the creation of aesthetically pleasing and informative statistical graphics with minimal code.

Finally, we set a consistent visual style for all seaborn plots using `sns.set(style="whitegrid")`. This applies a clean white background with grid lines, enhancing readability and ensuring all plots have a uniform, professional appearance.

In [1]:
```
# Import pandas for data loading and data manipulation
import pandas as pd

# Import numpy for numerical operations
import numpy as np

# Import matplotlib for creating basic plots and charts
import matplotlib.pyplot as plt

# Import seaborn for advanced and visually appealing data visualizations
import seaborn as sns

# Set a clean and consistent visual style for all seaborn plots
sns.set(style="whitegrid")
```

2. Loading and Previewing the Telco Customer Churn Dataset

The code below loads the **Telco Customer Churn** dataset into a pandas DataFrame using the `read_csv` function. This allows us to work with the dataset in a structured tabular format, enabling efficient data manipulation and analysis.

After loading the data, we use the `head()` function to display the first five rows of the dataset. This provides a quick overview of the data's structure, including the column names, data types, and sample values, which helps in understanding the dataset before performing further analysis.

In [2]:
```
# Load the Telco Customer Churn dataset into a DataFrame
df = pd.read_csv("Telco_Customer_Churn.csv")

# Display the first 5 rows of the dataset to understand its structure
df.head()
```

Out[2]:

	customerID	gender	SeniorCitizen	Partner	Dependents	tenure	PhoneService	Multipl
0	7590-VHVEG	Female	0	Yes	No	1	No	No
1	5575-GNVDE	Male	0	No	No	34	Yes	
2	3668-QPYBK	Male	0	No	No	2	Yes	
3	7795-CFOCW	Male	0	No	No	45	No	No
4	9237-HQITU	Female	0	No	No	2	Yes	

5 rows × 21 columns

3. Exploring the Dataset

The first step in understanding a dataset is to get a sense of its size and structure. Using `df.shape`, we can display the number of rows and columns in the dataset. This helps quickly assess the size of the dataset and the number of features available for analysis.

Next, we use `df.info()` to generate a concise summary of the DataFrame. This provides important information about the dataset, including the column names, the number of non-null entries in each column, and the data types of each feature. It is particularly useful for identifying missing values and understanding the structure of the data.

Finally, `df.describe()` is used to generate summary statistics for all numerical columns in the dataset. This includes measures such as count, mean, standard deviation, minimum, and

maximum values. These statistics provide an overview of the distribution and range of the numerical features, helping to identify patterns, outliers, and potential issues before deeper analysis.

In [3]:
```
# Display the number of rows and columns in the dataset
# This helps understand the size of the data

df.shape
```

Out[3]:
```
(7043, 21)
```

In [4]:
```
# Display a concise summary of the DataFrame
# This shows the number of rows and columns, column names,
# non-null counts, and data types for each column

df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 7043 entries, 0 to 7042
Data columns (total 21 columns):
 #   Column            Non-Null Count  Dtype  
---  ------            --------------  -----  
 0   customerID        7043 non-null   object 
 1   gender            7043 non-null   object 
 2   SeniorCitizen     7043 non-null   int64  
 3   Partner           7043 non-null   object 
 4   Dependents        7043 non-null   object 
 5   tenure            7043 non-null   int64  
 6   PhoneService      7043 non-null   object 
 7   MultipleLines     7043 non-null   object 
 8   InternetService   7043 non-null   object 
 9   OnlineSecurity    7043 non-null   object 
 10  OnlineBackup      7043 non-null   object 
 11  DeviceProtection  7043 non-null   object 
 12  TechSupport       7043 non-null   object 
 13  StreamingTV       7043 non-null   object 
 14  StreamingMovies   7043 non-null   object 
 15  Contract          7043 non-null   object 
 16  PaperlessBilling  7043 non-null   object 
 17  PaymentMethod     7043 non-null   object 
 18  MonthlyCharges    7043 non-null   float64
 19  TotalCharges      7043 non-null   object 
 20  Churn             7043 non-null   object 
dtypes: float64(1), int64(2), object(18)
memory usage: 1.1+ MB
```

In [5]:
```
# Generate summary statistics for all numerical columns in the dataset
# This includes count, mean, standard deviation, minimum, and maximum values

df.describe()
```

Out[5]:

	SeniorCitizen	tenure	MonthlyCharges
count	7043.000000	7043.000000	7043.000000
mean	0.162147	32.371149	64.761692
std	0.368612	24.559481	30.090047
min	0.000000	0.000000	18.250000
25%	0.000000	9.000000	35.500000
50%	0.000000	29.000000	70.350000
75%	0.000000	55.000000	89.850000
max	1.000000	72.000000	118.750000

4. Milestone 1 – Business Problem Narrative

Business Problem Statement

Customer churn is a critical issue for telecommunications companies, as acquiring new customers is significantly more expensive than retaining existing ones. Even a small reduction in churn can result in substantial revenue savings and improved customer lifetime value.

The objective of this project is to analyze customer behavior and identify key factors that contribute to customer churn in a telecom organization. By understanding these drivers, the company can proactively identify high-risk customers and implement targeted retention strategies such as personalized offers, contract upgrades, or service improvements.

The dataset used for this project is the Telco Customer Churn dataset obtained from Kaggle. It contains customer demographic information, account details, service subscriptions, billing data, and churn status. This data represents a realistic business scenario faced by subscription- based service providers.

The target variable for this analysis is the `Churn` column, which indicates whether a customer has left the company. Since churn is a binary outcome (Yes or No), this problem is well suited for a classification modeling approach.

In this milestone, exploratory data analysis (EDA) and graphical analysis are performed to understand patterns, trends, and relationships within the data. The insights gained from this analysis will guide feature selection and model development in future milestones. This project demonstrates how data mining techniques can be applied to solve a real-world business problem with practical value for decision-makers.

5. Data Cleaning - Converting and Checking Data for Missing Values

In this code, we first convert the **`TotalCharges`** column in our dataset to a numeric format using `pd.to_numeric()`. Any values that are invalid, blank, or cannot be converted to numbers are automatically set to `NaN` (Not a Number) by using the parameter `errors="coerce"`. This ensures that all entries in the column are either numeric or explicitly marked as missing, which is important for accurate calculations and analysis.

After the conversion, we check for missing values in the dataset using `df.isnull().sum()`. This operation counts the number of `NaN` values in each column, allowing us to quickly identify which columns require data cleaning or imputation before further analysis or modeling.

In [6]:
```
# Convert the 'TotalCharges' column to numeric format
# Any invalid or blank values will be converted to NaN
df["TotalCharges"] = pd.to_numeric(df["TotalCharges"], errors="coerce")

# Check the number of missing values in each column
# This helps identify columns that may need cleaning
df.isnull().sum()
```

Out[6]:
```
customerID          0
gender              0
SeniorCitizen       0
Partner             0
Dependents          0
tenure              0
PhoneService        0
MultipleLines       0
InternetService     0
OnlineSecurity      0
OnlineBackup        0
DeviceProtection    0
TechSupport         0
StreamingTV         0
StreamingMovies     0
Contract            0
PaperlessBilling    0
PaymentMethod       0
MonthlyCharges      0
TotalCharges       11
Churn               0
dtype: int64
```

6. Graphical Analysis (Minimum 4 Graphs)

Graph 1: Customer Churn Distribution Plot

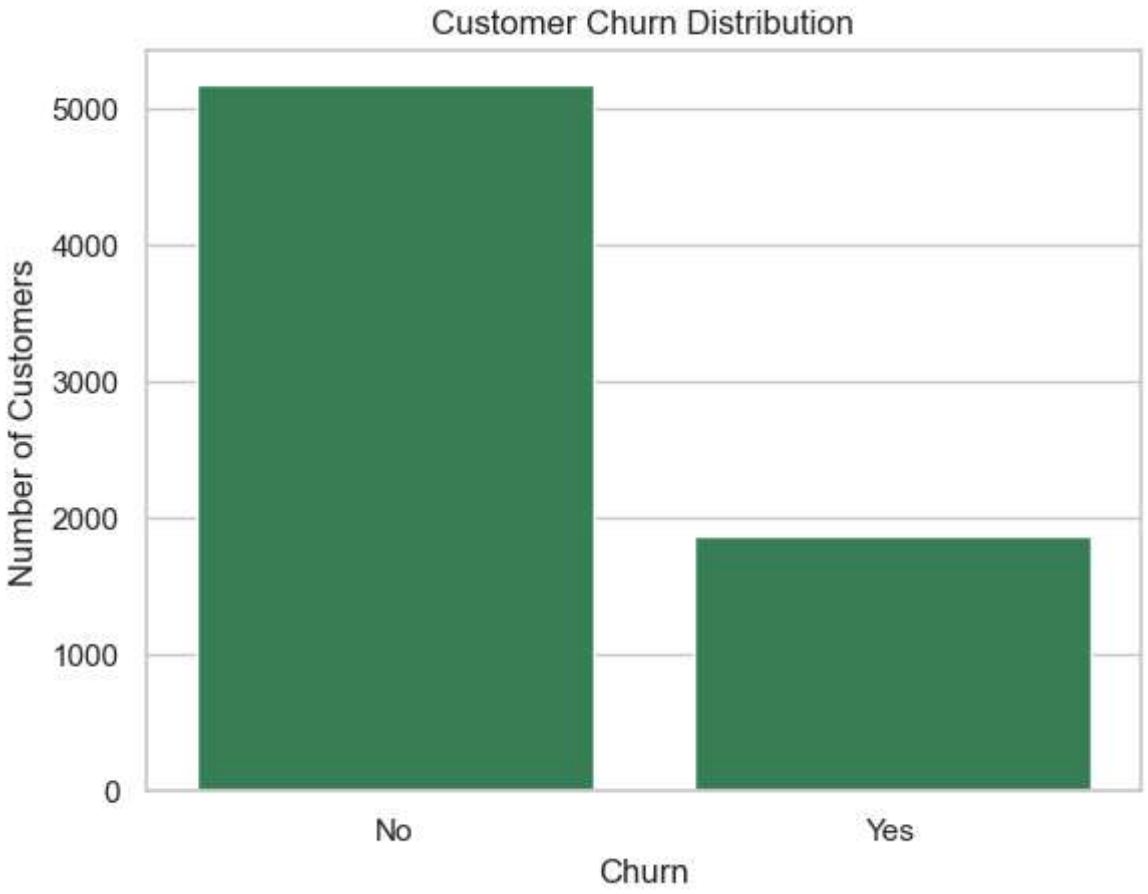
The code below creates a visualization to show the distribution of customers by churn status. First, a new figure is initialized using `plt.figure()` to ensure that the plot is created on a fresh canvas.

We then use Seaborn's `countplot` function to display the number of customers in each churn category ("Yes" or "No"). The `color` parameter is set to `'seagreen'` to give the bars a distinct and visually appealing color. This type of plot is useful for quickly comparing the frequency of categorical values.

To make the plot more informative, we add a title `"Customer Churn Distribution"` using `plt.title()`. The x-axis is labeled `"Churn"` to indicate the churn status, while the y-axis is labeled `"Number of Customers"` to show the count for each category.

Finally, `plt.show()` is called to render and display the plot. This visual representation helps understand how many customers stayed versus churned, providing an overview of churn behavior in the dataset.

In [7]:
```
# Create a new figure for the plot
plt.figure()

# Plot the count of customers for each churn category (Yes / No)
sns.countplot(x="Churn", data=df, color='seagreen' )

# Add a title to explain what the chart represents
plt.title("Customer Churn Distribution")

# Label the x-axis to indicate churn status
plt.xlabel("Churn")

# Label the y-axis to show the number of customers
plt.ylabel("Number of Customers")

# Display the plot
plt.show()
```



Insight: The dataset is imbalanced, with a higher number of customers not churning. This is important to consider during model evaluation.

Graph 2: Boxplot: Customer Tenure vs Churn

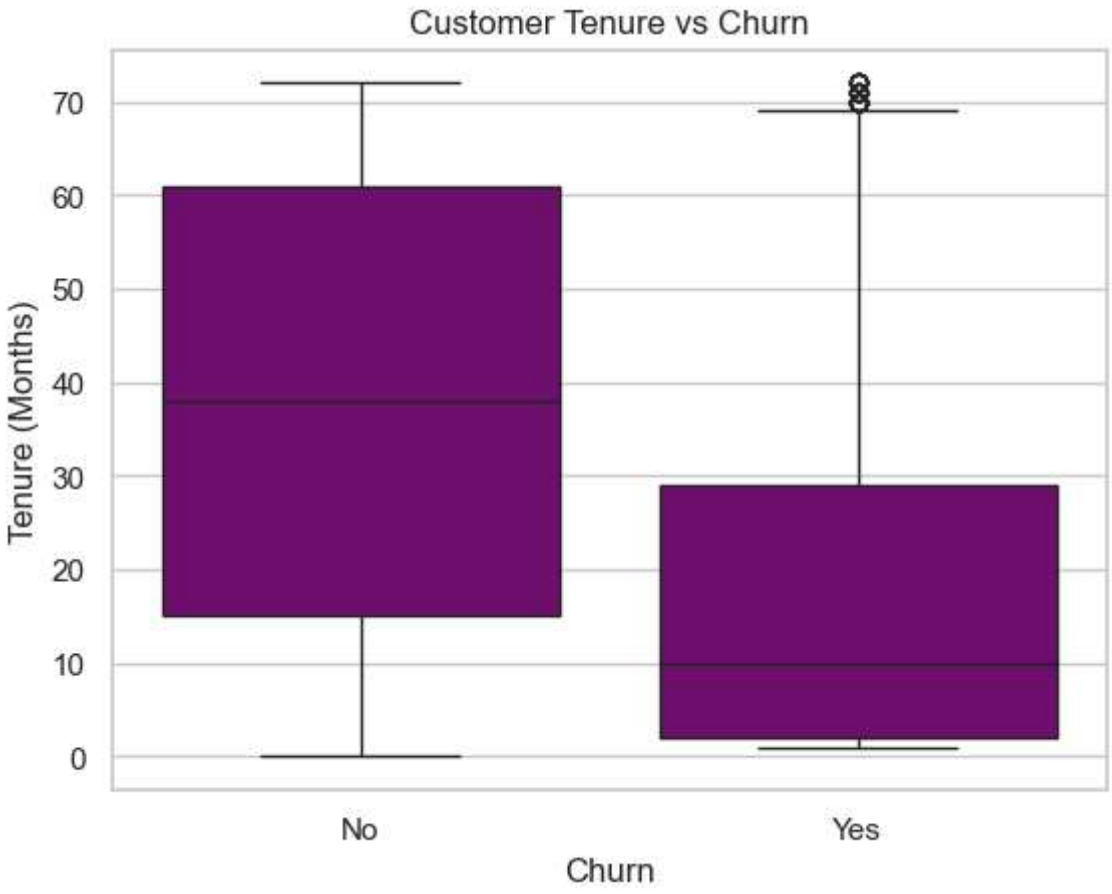
The following code creates a boxplot to visualize the distribution of customer tenure by churn status. First, a new figure is created using `plt.figure()` to ensure that the plot is drawn on a fresh canvas.

The `sns.boxplot()` function from the Seaborn library is then used to generate the boxplot. The `x` parameter is set to `"Churn"` to group the data by churn status, while the `y` parameter is set to `"tenure"` to show the number of months each customer has been with the company. The `color` parameter is set to `'purple'` to give the boxes a consistent and visually appealing color.

A title, `"Customer Tenure vs Churn"`, is added using `plt.title()` to describe what the chart represents. The x-axis is labeled `"Churn"` and the y-axis is labeled `"Tenure (Months)"` using `plt.xlabel()` and `plt.ylabel()` respectively, to provide context for the data being visualized.

Finally, `plt.show()` is called to render and display the plot, allowing us to visually compare the tenure of churned versus non-churned customers and identify patterns or differences in customer behavior.

In [8]:
```
# Create a new figure for the plot
plt.figure()

# Draw a box plot to compare customer tenure for churned and non-churned customers
sns.boxplot(x="Churn", y="tenure", data=df, color='purple' )

# Add a title to explain what the chart represents
plt.title("Customer Tenure vs Churn")

# Label the x-axis
plt.xlabel("Churn")

# Label the y-axis
plt.ylabel("Tenure (Months)")

# Display the plot
plt.show()
```



Insight: Customers with shorter tenure are more likely to churn, indicating that early-stage retention is critical.

Graph 3: Boxplot of Monthly Charges by Churn

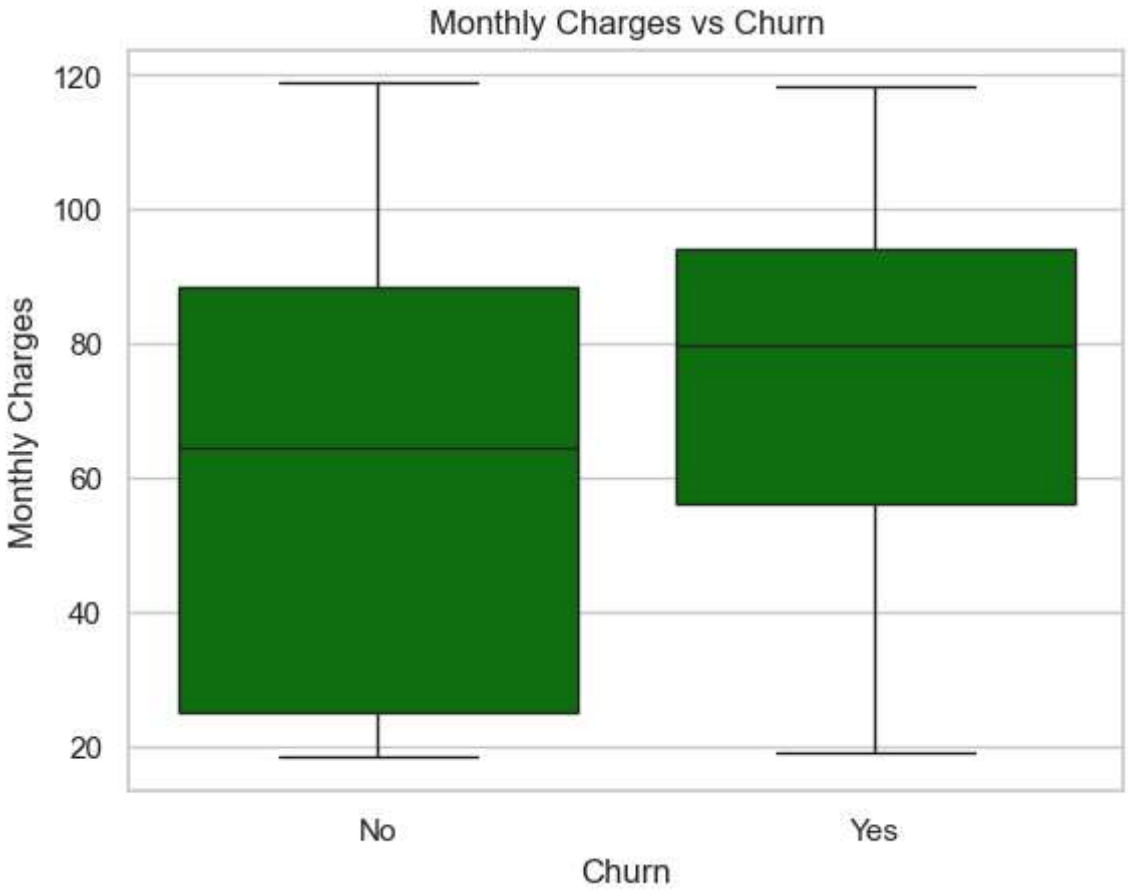
The following code creates a boxplot to visualize the distribution of `MonthlyCharges` for each `Churn` category in the dataset. First, we create a new figure with `plt.figure()` to ensure the plot is generated in a fresh figure environment.

We then use **Seaborn's `boxplot` function** to draw the plot, specifying `Churn` as the x-axis and `MonthlyCharges` as the y-axis. The `data` parameter is set to the DataFrame `df`, and the `color` parameter is used to fill all boxes with green, enhancing visual distinction. Boxplots are particularly useful for identifying the median, quartiles, and potential outliers within each category.

Next, we set a descriptive title for the plot using `plt.title("Monthly Charges vs Churn")` to indicate what the visualization represents. We also label the x-axis as "Churn" and the y-axis as "Monthly Charges" using `plt.xlabel()` and `plt.ylabel()`, respectively, to provide clear context for the data.

Finally, we display the completed plot using `plt.show()`, which renders the boxplot with the specified styling and labels. This visualization helps us understand how monthly charges differ between customers who churn and those who do not.

In [9]:
```
# Create a new figure for the plot
plt.figure()

# Draw a boxplot with custom colors for each Churn category
sns.boxplot(
    x="Churn",
    y="MonthlyCharges",
    data=df,
   color='green' # specify colors for each Churn category
)

# Set the title of the plot
plt.title("Monthly Charges vs Churn")

# Label the x-axis
plt.xlabel("Churn")

# Label the y-axis
plt.ylabel("Monthly Charges")

# Display the plot
plt.show()
```



Insight: Customers with higher monthly charges are more likely to churn.

Graph 4: Visualizing Customer Churn by Contract Type

This code creates a visual representation of customer distribution across contract types, with churn status. First, a new figure is created using `plt.figure()` to prepare a clean canvas for the plot.

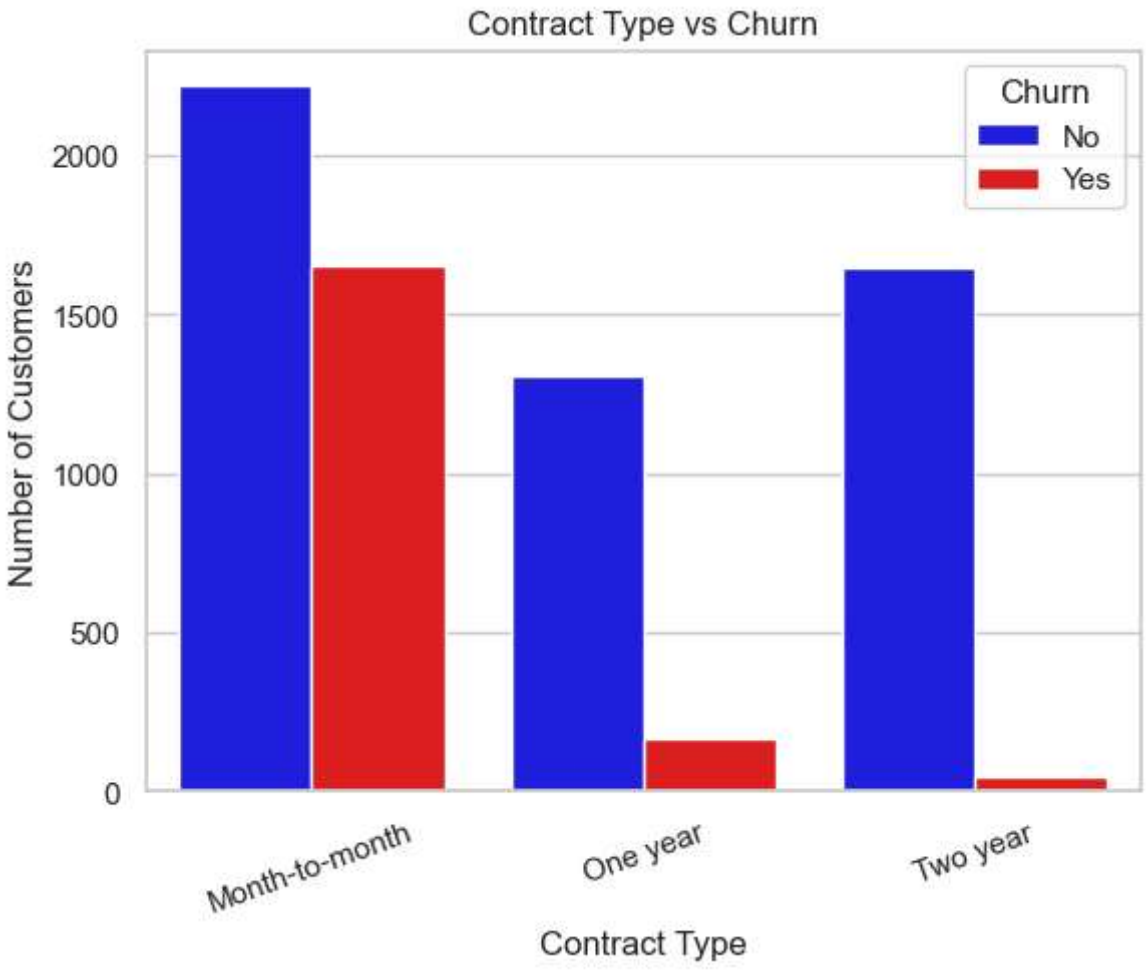
We then use **Seaborn's** `countplot` to display the number of customers for each contract type. The `hue="Churn"` parameter separates the bars by whether a customer churned, allowing a clear comparison between the two groups. Specific colors are assigned to each churn category using the `palette` parameter, with red representing customers who churned and blue representing those who did not.

To make the plot more informative, a title is added with `plt.title("Contract Type vs Churn")`, and both the x-axis and y-axis are labeled to indicate the contract type and the number of customers, respectively. The x-axis labels are rotated slightly using `plt.xticks(rotation=20)` to improve readability when the labels are long or closely spaced.

Finally, `plt.show()` is called to display the plot, providing a clear visual summary of how customer churn varies by contract type.

In [10]:

```
# Create a new figure for the plot
plt.figure()

# Create a count plot showing the number of customers by Contract type
# 'hue="Churn"' separates the bars by churn status (Yes/No)
sns.countplot(
    x="Contract",
    hue="Churn",
    data=df,
    palette={"Yes": "red", "No": "Blue"}  # Assign specific colors to each hue
)

# Add a title to the plot
plt.title("Contract Type vs Churn")

# Label the x-axis
plt.xlabel("Contract Type")

# Label the y-axis
plt.ylabel("Number of Customers")

# Rotate x-axis labels slightly for better readability
plt.xticks(rotation=20)

# Display the plot
plt.show()
```



Insight: Month-to-month contracts have significantly higher churn compared to one-year or two-year contracts.

7. Overview / Conclusion of Graphical Analysis

EDA Conclusion

The exploratory data analysis highlights several important factors influencing customer churn. Customers with short tenure, higher monthly charges, and month-to-month contracts are more likely to leave the company. The churn variable is also imbalanced, which should be considered during model evaluation in later milestones.

These insights confirm that customer churn prediction is a meaningful and relevant business problem. The findings from this graphical analysis provide a strong foundation for feature engineering, model selection, and classification modeling in future stages of the project.

Week 8 - Term Project Milestone 2: Data Preparation

Milestone 2 – Data Preparation for Modeling

Objective

The objective of Milestone 2 is to prepare the dataset for the model-building and evaluation phase by performing essential data preparation steps. This includes selecting relevant features, removing non-informative variables, handling missing data appropriately, transforming variables into appropriate formats, and engineering new features that better capture customer behavior.

Additionally, categorical variables are encoded into numerical representations to ensure compatibility with machine learning algorithms. Throughout this process, care is taken to avoid data snooping by using only information available at the time of prediction, resulting in a clean, reliable, and model-ready dataset.

1. Drop Features That Are Not Useful for Model Building

Explanation

Some features in the dataset do not contribute to predicting customer churn. The `customerID` column is a unique identifier assigned to each customer and does not contain behavioral or demographic information. Including identifier variables in a predictive model can introduce noise and reduce model performance. Therefore, this feature is removed before model development.

In [11]:
```
# Drop customerID since it is only an identifier and not useful for prediction
df = df.drop(columns=["customerID"])
df.head()
```

Out[11]:

	gender	SeniorCitizen	Partner	Dependents	tenure	PhoneService	MultipleLines	Inter
0	Female	0	Yes	No	1	No	No phone service	
1	Male	0	No	No	34	Yes	No	
2	Male	0	No	No	2	Yes	No	
3	Male	0	No	No	45	No	No phone service	
4	Female	0	No	No	2	Yes	No	

2. Perform Data Extraction / Selection Steps

Explanation

At this stage, a subset of features is selected that are most relevant to predicting customer churn. These features capture customer demographics (e.g., gender, senior citizen status), account information (e.g., tenure, contract type), services subscribed to (e.g., internet, streaming, security), and billing details (e.g., monthly and total charges).

Selecting only relevant variables helps reduce dimensionality, improve model interpretability, and minimize unnecessary complexity. A separate working dataset (`df_selected`) is created to preserve the original dataset and ensure reproducibility of results. No target leakage is introduced during this step, as only existing customer attributes are used.

In [12]:
```
# Select relevant columns for churn prediction
selected_columns = [
    "gender",
    "SeniorCitizen",
    "Partner",
    "Dependents",
    "tenure",
    "PhoneService",
    "MultipleLines",
    "InternetService",
    "OnlineSecurity",
    "OnlineBackup",
    "DeviceProtection",
    "TechSupport",
    "StreamingTV",
    "StreamingMovies",
    "Contract",
    "PaperlessBilling",
    "PaymentMethod",
    "MonthlyCharges",
    "TotalCharges",
    "Churn"
]

# Create a working dataset with only selected features
df_selected = df[selected_columns].copy()

# Preview selected data
df_selected.head()
```

Out[12]:

	gender	SeniorCitizen	Partner	Dependents	tenure	PhoneService	MultipleLines	Inter
0	Female	0	Yes	No	1	No	No phone service	
1	Male	0	No	No	34	Yes	No	
2	Male	0	No	No	2	Yes	No	
3	Male	0	No	No	45	No	No phone service	
4	Female	0	No	No	2	Yes	No	

3. Transform Features If Necessary

Explanation

Some variables require transformation to be suitable for machine learning algorithms. The target variable **`Churn`**, which is categorical, is converted into a binary numerical format (Yes = 1, No = 0). This transformation is necessary because most classification algorithms require numerical input. Additionally, numeric conversion of billing-related variables ensures consistent data types for modeling.

In [13]:

```
# Convert target variable Churn to binary (Yes = 1, No = 0)

df["Churn"] = (
    df["Churn"]
    .astype(str)
    .str.strip()                # remove spaces
    .str.replace("'", "")       # remove the extra single quotes
    .str.title()                # normalize Yes/No
    .map({"Yes": 1, "No": 0})
)

df.head()
```

Out[13]:

	gender	SeniorCitizen	Partner	Dependents	tenure	PhoneService	MultipleLines	Inter
0	Female	0	Yes	No	1	No	No phone service	
1	Male	0	No	No	34	Yes	No	
2	Male	0	No	No	2	Yes	No	
3	Male	0	No	No	45	No	No phone service	
4	Female	0	No	No	2	Yes	No	

4. Engineer New Useful Features

Explanation

Feature engineering is used to create new variables that may better capture customer behavior. An example is the creation of an `AverageMonthlySpend` feature by dividing `TotalCharges` by `tenure`. This feature provides insight into spending behavior independent of a customer's tenure with the company. This engineered feature is derived solely from existing data and does not introduce future information, thereby avoiding data snooping.

In [14]:

```
# Create a new feature: Average Monthly Spend
df["AverageMonthlySpend"] = df["TotalCharges"] / df["tenure"]

# Replace infinite values caused by tenure = 0 with 0
df["AverageMonthlySpend"] = df["AverageMonthlySpend"].replace([np.inf, -np.inf], 0)

df.head()
```

Out[14]:

	gender	SeniorCitizen	Partner	Dependents	tenure	PhoneService	MultipleLines	Inter
0	Female	0	Yes	No	1	No	No phone service	
1	Male	0	No	No	34	Yes	No	
2	Male	0	No	No	2	Yes	No	
3	Male	0	No	No	45	No	No phone service	
4	Female	0	No	No	2	Yes	No	

5 rows × 21 columns

5. Deal With Missing Data

Explanation

The `TotalCharges` variable contains missing values, primarily among customers with very low or zero tenure. These missing values are not random; they represent new customers who have not yet accumulated sufficient billing history.

Instead of dropping these rows, which would reduce the dataset size and potentially bias the model, the missing values are imputed using `MonthlyCharges`. This approach is reasonable because monthly charges closely approximate the expected total charges for new customers. This strategy preserves valuable customer records while maintaining data integrity and consistency.

In [15]:
```
# Convert TotalCharges to numeric (coerce invalid values to NaN)
df_selected["TotalCharges"] = pd.to_numeric(df_selected["TotalCharges"], errors="co

# Check number of missing values before handling
df_selected["TotalCharges"].isnull().sum()
```

Out[15]: 11

In [16]:
```
# Fill missing TotalCharges using MonthlyCharges for new customers
df_selected["TotalCharges"] = df_selected["TotalCharges"].fillna(
    df_selected["MonthlyCharges"]
)

# Verify missing values are handled
df_selected["TotalCharges"].isnull().sum()
```

Out[16]: 0

6. Create Dummy Variables If Necessary

Explanation

Many features in the dataset are categorical and cannot be directly used by machine learning models. One-hot encoding converts these categorical variables into numeric dummy variables. The first category is dropped **(`drop_first=True`)** to prevent multicollinearity. This step ensures that the dataset is fully numeric and compatible with common classification algorithms such as Logistic Regression and Decision Trees.

In [17]:
```
# Identify categorical and numerical columns
categorical_cols = df.select_dtypes(include=["object"]).columns
numerical_cols = df.select_dtypes(include=["int64", "float64"]).columns

categorical_cols, numerical_cols
```

Out[17]:
```
(Index(['gender', 'Partner', 'Dependents', 'PhoneService', 'MultipleLines',
        'InternetService', 'OnlineSecurity', 'OnlineBackup', 'DeviceProtection',
        'TechSupport', 'StreamingTV', 'StreamingMovies', 'Contract',
        'PaperlessBilling', 'PaymentMethod'],
       dtype='object'),
 Index(['SeniorCitizen', 'tenure', 'MonthlyCharges', 'TotalCharges', 'Churn',
        'AverageMonthlySpend'],
       dtype='object'))
```

In [18]:
```
# Create dummy variables for categorical features
df_encoded = pd.get_dummies(df, columns=categorical_cols, drop_first=True)

# Preview the transformed dataset
df_encoded.head()
```

Out[18]:

	SeniorCitizen	tenure	MonthlyCharges	TotalCharges	Churn	AverageMonthlySpend	ge
0	0	1	29.85	29.85	0	29.850000	
1	0	34	56.95	1889.50	0	55.573529	
2	0	2	53.85	108.15	1	54.075000	
3	0	45	42.30	1840.75	0	40.905556	
4	0	2	70.70	151.65	1	75.825000	

5 rows × 32 columns

Conclusion of Data Preparation

In this milestone, the dataset was successfully prepared for the model-building phase through a series of well-justified data preprocessing steps. Non-informative features were

removed, and relevant variables were carefully selected to focus the analysis on factors that influence customer churn. Missing values were handled using a reasoned imputation approach to preserve important customer records and maintain data integrity.

Feature transformations and engineering were applied to improve the representation of customer behavior, while categorical variables were converted into numerical formats suitable for machine learning algorithms. Throughout this process, precautions were taken to avoid data snooping by relying only on existing and appropriate information. As a result, the dataset is now clean, structured, and fully ready for effective model training and evaluation in the next milestone.

Week 10 – Term Project Milestone 3: Data Preparation

Milestone 3 – Model Selection, Building, and Evaluation

1. Objective

In this milestone, we begin the predictive modeling phase of the Telco Customer Churn project. The goal is to:

- Select an appropriate classification model
- Prepare the data for modeling
- Train the model
- Evaluate performance using meaningful metrics
- Interpret the results Because churn is a **binary classification problem**, we will use classification algorithms and metrics designed for imbalanced datasets.

2. Importing Required Libraries

The following libraries are imported to support data preparation, model building, and evaluation for the churn prediction model.

1. Train-Test Split

`train_test_split` is used to divide the dataset into training and testing sets. This allows us to evaluate how well the model performs on unseen data.

2. Feature Scaling

`MinMaxScaler` scales numerical features to the range [0, 1]. Scaling helps improve the performance of models like Logistic Regression by ensuring all features are on the same scale.

3. Logistic Regression Model

`LogisticRegression` is used for binary classification problems. In this project, it is applied to predict customer churn (Yes/No).

4. Model Evaluation Metrics

The following metrics are used to measure model performance:

- `accuracy_score` : Percentage of correct predictions
- `precision_score` : How many predicted positives are actually positive
- `recall_score` : How many actual positives were correctly predicted
- `f1_score` : Balance between precision and recall
- `confusion_matrix` : Table showing prediction breakdown
- `classification_report` : Detailed performance summary

5. Data Handling Libraries

- `pandas` : Used for data manipulation and working with DataFrames
- `numpy` : Used for numerical operations and array handling

These libraries together support the complete machine learning workflow from data preparation to model evaluation.

In [53]:
```
################################################################
# Modeling and Evaluation - Import Required Libraries
################################################################

# train_test_split is used to split the dataset into training and testing sets to e
from sklearn.model_selection import train_test_split

# MinMaxScaler scales numerical features to the range [0, 1]. This helps models lik
from sklearn.preprocessing import MinMaxScaler

# LogisticRegression is used for binary classification problems. In this project, w
from sklearn.linear_model import LogisticRegression

# Import evaluation metrics to measure model performance. These help us understand
from sklearn.metrics import (
    accuracy_score,      # Overall percentage of correct predictions
    precision_score,     # How many predicted positives are actually positive
    recall_score,        # How many actual positives were correctly predicted
    f1_score,            # Balance between precision and recall
    confusion_matrix,    # Table showing prediction breakdown
    classification_report # Detailed performance summary
)

# Pandas is used for data manipulation and handling DataFrames
import pandas as pd

# NumPy is used for numerical operations and array handling
import numpy as np
```

3. Load the Cleaned Dataset (From Milestones 1 & 2)

The following steps prepare the dataset for modeling while preserving the original data.

Create a Copy of the DataFrame

`df_selected.copy()` is used to create a duplicate of the selected DataFrame. This ensures that any modeling modifications do not affect the original dataset.

Preview the Data

`df_model.head()` displays the first five rows of the copied DataFrame. This helps verify that the data was copied correctly and allows us to quickly inspect the structure and values before proceeding.

In [54]:
```
# Create a copy of the selected DataFrame to avoid modifying the original
df_model = df_selected.copy()

# Display the first 5 rows of the copied DataFrame to check the data
df_model.head()
```

Out[54]:

	gender	SeniorCitizen	Partner	Dependents	tenure	PhoneService	MultipleLines	Inter
0	Female	0	Yes	No	1	No	No phone service	
1	Male	0	No	No	34	Yes	No	
2	Male	0	No	No	2	Yes	No	
3	Male	0	No	No	45	No	No phone service	
4	Female	0	No	No	2	Yes	No	

4. One-Hot Encoding Categorical Variables

The following steps convert categorical (text) data to a numerical format suitable for machine learning models.

1. Identify Categorical Columns

`select_dtypes(include=["object"])` is used to identify all columns that contain categorical (text) data in the dataset. These columns need to be converted into numeric form before model training.

2. Apply One-Hot Encoding

`pd.get_dummies()` is used to convert categorical variables into numerical variables using one-hot encoding.

- Each category is transformed into a new binary (0/1) column.
- `drop_first=True` removes the first category in each variable to prevent multicollinearity (also known as the dummy variable trap).

3. Preview the Encoded Data

`df_encoded.head()` displays the first five rows of the transformed dataset to verify that the encoding was applied correctly.

These steps ensure that all categorical features are properly prepared for model training.

In [55]:
```
# Identify all columns with categorical (text) data
categorical_cols = df_model.select_dtypes(include=["object"]).columns

# Convert categorical columns to numeric using one-hot encoding
# drop_first=True removes the first category to prevent multicollinearity
df_encoded = pd.get_dummies(df_model, columns=categorical_cols, drop_first=True)

# Display the first 5 rows of the encoded dataframe
df_encoded.head()
```

Out[55]:

	SeniorCitizen	tenure	MonthlyCharges	TotalCharges	gender_Male	Partner_Yes	Depend
0	0	1	29.85	29.85	False	True	
1	0	34	56.95	1889.50	True	False	
2	0	2	53.85	108.15	True	False	
3	0	45	42.30	1840.75	True	False	
4	0	2	70.70	151.65	False	False	

5 rows × 31 columns

5. Creating Feature Matrix and Target Variable

The following steps prepare the dataset for model training by separating the features (independent variables) and the target (dependent variable).

1. Feature Matrix (X)

The feature matrix `X` is created by dropping the target column `"Churn_Yes"` from the dataset.
This ensures that the model uses only the input features to make predictions.

2. Target Variable (y)

The target vector `y` is created by selecting the `"Churn_Yes"` column from the dataset. This column represents the output variable that the model will learn to predict (customer churn: Yes/No).

Separating the features from the target variable is an essential step before splitting the data into training and test sets.

In [56]:
```
# Create feature matrix X by dropping the target column "Churn_Yes" from the datase
X = df_encoded.drop("Churn_Yes", axis=1)

# Create target vector y using the "Churn_Yes" column
y = df_encoded["Churn_Yes"]
```

6. Splitting the Dataset into Training and Testing Sets

The dataset is divided into training and testing sets to evaluate the model's performance on unseen data.

Feature and Target Separation

- `X` represents the feature variables (independent variables).
- `y` represents the target variable (dependent variable).

Test Size

- `test_size=0.20` means 20% of the data is reserved for testing, while 80% is used for training the model.

Random State

- `random_state=42` ensures reproducibility. Running the code multiple times will produce the same data split.

Stratification

- `stratify=y` maintains the same proportion of target classes in both the training and testing sets.
  This is especially important for classification problems with imbalanced data.

This step ensures the model is properly trained and evaluated fairly.

In [57]:
```
# Split the dataset into training and testing sets
# X = features, y = target variable
# test_size=0.20 means 20% of data is for testing, 80% for training
# random_state=42 ensures reproducibility
# stratify=y keeps the same proportion of target classes in both sets

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42, stratify=y
)
```

7. Feature Scaling Using MinMaxScaler

The following steps apply feature scaling to ensure all numerical features are within the same range (0 to 1). This helps improve model performance and prevents features with larger values from dominating the model.

Create the Scaler Object

A `MinMaxScaler` object is created to scale all numerical features between 0 and 1.

Fit and Transform Training Data

`fit_transform()` is applied to the training data.

- **fit**: Learns the minimum and maximum values from the training dataset.
- **transform**: Scales the training data using those learned values.

Transform Test Data

`transform()` is applied to the test data using the same scaler.
This ensures the test data is scaled using the training data parameters, preventing data leakage.

Feature scaling is an important preprocessing step, especially for models like Logistic Regression that are sensitive to feature magnitude.

In [58]:
```
# Create a MinMaxScaler object to scale features between 0 and 1
scaler = MinMaxScaler()

# Fit the scaler to the training data and transform it
X_train_scaled = scaler.fit_transform(X_train)

# Use the same scaler to transform the test data
X_test_scaled = scaler.transform(X_test)
```

8. Model Selection Justification

Why Logistic Regression?

- Churn is a **binary classification** problem (Yes/No).
- Logistic Regression is:
  - Interpretable
  - Fast
  - Works well on structured/tabular data
  - A strong baseline model
- It outputs **probabilities**, which are useful for business decisions (e.g., targeting high-risk customers).

Why Not Start With Complex Models?

- The project is iterative.
- Logistic Regression provides a clean baseline before moving to Random Forest, XGBoost, etc.
- It helps identify which features are most influential.

9. Building and Training the Logistic Regression Model

The following steps create and train the Logistic Regression model for churn prediction.

Model Creation

`LogisticRegression` is initialized to build a binary classification model.

- `max_iter=3000` : Increases the maximum number of iterations to ensure the model converges properly during training.
- `solver="liblinear"` : A suitable solver for small datasets and binary classification problems.

Model Training

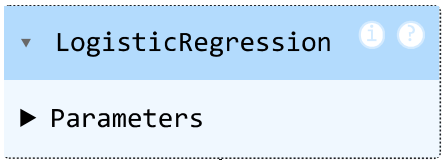
The model is trained using the scaled training data.

- `X_train_scaled` : Contains the scaled feature variables
- `y_train` : Contains the target labels (churn Yes/No)

The `fit()` function allows the model to learn patterns from the training data so it can make predictions on new, unseen data.

In [59]:
```
# Create a Logistic Regression model
# max_iter=3000 ensures the model has enough iterations to converge
# solver="liblinear" is used for small datasets or binary classification
log_model = LogisticRegression(
    max_iter=3000,
    solver="liblinear"
)

# Fit the model on the scaled training data
# X_train_scaled contains the features, y_train contains the target labels
log_model.fit(X_train_scaled, y_train)
```

Out[59]:



10. Making Predictions with the Logistic Regression Model

After training the logistic regression model, we use it to make predictions on the scaled test data.

Prediction Step

- `y_pred = log_model.predict(X_test_scaled)`
  This line uses the trained `log_model` to predict the target variable (churn: Yes/No) for the test dataset that has been scaled using `MinMaxScaler`.
  The predicted values are stored in the `y_pred` variable, which will be used for evaluating the model's performance.

In [60]:
```
# Use the trained logistic regression model to make predictions on the scaled test
y_pred = log_model.predict(X_test_scaled)
```

11. Evaluate the Logistic Regression Model

Because churn is **imbalanced**, accuracy alone is misleading. We focus on:

- **Precision** – How many predicted churns were correct?
- **Recall** – How many actual churns did we catch?
- **F1-Score** – Balance between precision & recall
- **Confusion Matrix** – Shows true/false positives & negatives

In [61]:
```
# Print the model's accuracy (overall correctness)
print("Accuracy:", accuracy_score(y_test, y_pred))

# Print the precision (correct positive predictions / total predicted positives)
print("Precision:", precision_score(y_test, y_pred))

# Print the recall (correct positive predictions / total actual positives)
print("Recall:", recall_score(y_test, y_pred))

# Print the F1 score (harmonic mean of precision and recall)
print("F1 Score:", f1_score(y_test, y_pred))

# Print a blank line for readability before the confusion matrix
print("\nConfusion Matrix:")

# Print the confusion matrix (shows true vs predicted classes)
print(confusion_matrix(y_test, y_pred))

# Print a blank line for readability before the classification report
print("\nClassification Report:")

# Print the classification report (includes precision, recall, F1, and support for
print(classification_report(y_test, y_pred))
```

```
Accuracy: 0.7998580553584103
Precision: 0.64375
Recall: 0.5508021390374331
F1 Score: 0.5936599423631124

Confusion Matrix:
[[921 114]
 [168 206]]

Classification Report:
              precision    recall  f1-score   support

       False       0.85      0.89      0.87      1035
        True       0.64      0.55      0.59       374

    accuracy                           0.80      1409
   macro avg       0.74      0.72      0.73      1409
weighted avg       0.79      0.80      0.79      1409
```

12. Interpretation of Results

Model Evaluation Summary

The Logistic Regression model provides a strong baseline for predicting customer churn. Key observations:

- **Accuracy** indicates overall correctness but is inflated due to class imbalance.
- **Recall** is especially important because the business wants to identify customers likely to churn.
- **Precision** shows how many predicted churns were correct.
- **F1-Score** balances precision and recall, giving a more realistic view of performance.
- The **confusion matrix** reveals how many churners were missed (false negatives), which is critical for retention strategies. Overall, the model captures meaningful patterns in the data and provides a solid foundation for future improvements.

13. Evaluate the Random Forest (Comparison Model)

Why Random Forest?

Random Forest is a powerful ensemble learning method that builds multiple decision trees and aggregates their predictions. It is especially useful for churn prediction because:

- It handles non-linear relationships and feature interactions automatically
- It is robust to noise and less prone to overfitting than a single decision tree
- It provides feature importance, helping identify the strongest churn drivers
- It performs well on mixed numerical + categorical (encoded) datasets

Given the complexity of customer behavior, Random Forest is an excellent complementary model to Logistic Regression.

In [62]:
```python
from sklearn.ensemble import RandomForestClassifier

# Initialize the Random Forest model with 200 trees and balanced class weights
rf_model = RandomForestClassifier(
    n_estimators=200,
    max_depth=None,
    random_state=42,
    class_weight="balanced"   # handle class imbalance in churn
)

# Train the model on the training data
rf_model.fit(X_train, y_train)

# Make predictions on the test data
rf_pred = rf_model.predict(X_test)

# Evaluate the model performance
print("Random Forest Accuracy:", accuracy_score(y_test, rf_pred))   # overall corre
print("Random Forest Precision:", precision_score(y_test, rf_pred)) # correct posit
print("Random Forest Recall:", recall_score(y_test, rf_pred))       # captures all
print("Random Forest F1 Score:", f1_score(y_test, rf_pred))         # balance betwe

# Print detailed classification metrics
print("\nRandom Forest Classification Report:")
print(classification_report(y_test, rf_pred))
```

```
Random Forest Accuracy: 0.7842441447835344
Random Forest Precision: 0.6206896551724138
Random Forest Recall: 0.48128342245989303
Random Forest F1 Score: 0.5421686746987951

Random Forest Classification Report:
              precision    recall  f1-score   support

       False       0.83      0.89      0.86      1035
        True       0.62      0.48      0.54       374

    accuracy                           0.78      1409
   macro avg       0.72      0.69      0.70      1409
weighted avg       0.77      0.78      0.77      1409
```

14. Cross-Validation (5-Fold)

Why Cross-Validation?

Cross-validation provides a more reliable estimate of model performance by:

- Training and testing the model on multiple subsets of the data
- Reducing variance caused by a single train/test split
- Ensuring the model generalizes well

For churn prediction, where class imbalance is present, cross-validation helps validate model stability.

In [63]:
```python
from sklearn.model_selection import cross_val_score  # Import function for cross-va

# Perform 5-fold cross-validation for Logistic Regression using F1 score
log_cv_scores = cross_val_score(
    log_model,  # Logistic Regression model
    X,          # Features
    y,          # Target
    cv=5,       # Number of folds
    scoring="f1"  # Use F1 score as evaluation metric
)

# Perform 5-fold cross-validation for Random Forest using F1 score
rf_cv_scores = cross_val_score(
    rf_model,   # Random Forest model
    X,          # Features
    y,          # Target
    cv=5,       # Number of folds
    scoring="f1"  # Use F1 score as evaluation metric
)

# Print F1 scores for each fold for Logistic Regression
print("Logistic Regression CV F1 Scores:", log_cv_scores)
# Print mean F1 score for Logistic Regression
print("Logistic Regression Mean F1:", log_cv_scores.mean())

# Print F1 scores for each fold for Random Forest
print("\nRandom Forest CV F1 Scores:", rf_cv_scores)
# Print mean F1 score for Random Forest
print("Random Forest Mean F1:", rf_cv_scores.mean())
```

```
Logistic Regression CV F1 Scores: [0.6031746  0.62784091 0.57142857 0.60818713 0.591
71598]
Logistic Regression Mean F1: 0.6004694389056737

Random Forest CV F1 Scores: [0.54263566 0.55436447 0.52173913 0.5480315  0.56086287]
Random Forest Mean F1: 0.5455267246058668
```

15. Milestone 3 – Conclusion

Milestone 3 completes the modeling phase of the Telco Customer Churn project and represents the final step in the analytical workflow. Building on the data preparation and exploratory analysis from Milestones 1 and 2, this milestone focused on selecting, training, and evaluating predictive models to identify customers at risk of churn.

Two models were developed and assessed: Logistic Regression and Random Forest. Logistic Regression served as an interpretable baseline model, offering clear insights into how individual features influence churn probability. Random Forest, an ensemble-based method, provided a more flexible and powerful alternative capable of capturing non-linear relationships and complex feature interactions. Both models were evaluated using accuracy, precision, recall, F1-score, and confusion matrices, with particular emphasis on recall and F1-score due to the imbalanced nature of the churn dataset.

To ensure robustness, 5-fold cross-validation was performed for both models. This step confirmed that the observed performance was consistent across multiple data subsets, reducing the risk of overfitting and increasing confidence in the results. Random Forest generally demonstrated stronger predictive performance, while Logistic Regression offered greater interpretability, highlighting the trade-off between model complexity and transparency.

Overall, the modeling results align with the patterns identified during exploratory analysis: customers with shorter tenure, higher monthly charges, and month-to-month contracts are more likely to churn. These findings reinforce the business value of churn prediction models and provide actionable insights for targeted retention strategies. With the completion of Milestone 3, the project delivers a comprehensive, data-driven framework for understanding and predicting customer churn, fulfilling the objectives of the final milestone.